Employee Attrition Prediction & Workplace Analytics

Project Workflow Report

A Concept-Level Walkthrough of the End-to-End Machine Learning Pipeline

Dataset: capstone_employee_attrition_dataset.csv | 20,000 records | 29 variables

1. Project Overview

This project builds a complete, end-to-end machine learning workflow around an HR dataset of 20,000 employees, with the goal of understanding and predicting employee attrition (whether an employee leaves the company or stays). The work moves through five connected stages: exploring the data, preparing it for modeling, building and evaluating classification models, segmenting employees through clustering, and, as a bonus, predicting income through regression.

The end objective mirrors what a real people-analytics team would do: flag employees who are at risk of leaving, understand what's driving that risk, and group employees into meaningful segments so retention efforts can be targeted rather than generic.

1.1 Business Problem

Losing an employee is expensive — replacing someone typically costs 50–200% of their annual salary once recruiting, onboarding, and lost productivity are factored in. If HR can spot at-risk employees early and understand why they're leaving, they can act before it's too late: adjusting pay, workload, or manager relationships for the people who need it most.

1.2 What the Project Delivers

- **A classification model** that predicts whether an employee will leave (Yes/No), along with accuracy, precision, recall, F1, and ROC-AUC scores.
- **An interpretability layer** that identifies the top features actually driving attrition, in plain business terms.
- **A clustering solution** that segments employees into natural groups, so we can see which segment carries the highest attrition risk.
- **A regression model** that predicts monthly income from an employee's profile, as a secondary exercise in understanding compensation drivers.

2. High-Level Workflow

The project follows a linear pipeline, where each stage feeds into the next. At a glance:

1. Load & Explore the Data (EDA)
2. Clean & Preprocess the Data
3. Split into Train and Test Sets
4. Train Classification Models
5. Evaluate & Interpret the Models
6. Segment Employees with Clustering
7. Predict Income with Regression
8. Summarize Findings & Recommendations

3. Step-by-Step Breakdown (Concept-Level)

3.1 Data Loading & Exploratory Data Analysis (EDA)

Before touching any model, the first job was simply to understand the data — its shape, its types, and whether anything was missing or broken.

Step	What We Did	Why It Matters
Load the dataset	Read the CSV into a DataFrame and checked its shape, column data types, and null counts.	Confirms the dataset is complete (it was — zero missing values) and tells us which columns are numeric versus categorical before we plan any preprocessing.
Check overall attrition rate	Calculated what fraction of the 20,000 employees left the company, and broke that rate down by department.	Gives a baseline: if ~16% of employees leave overall, our model needs to beat naive guessing, and we can see immediately which departments are hotspots.
Correlation analysis	Built a correlation matrix across all numeric features against a binary attrition flag, and ranked features by strength of correlation.	Surfaces which numeric variables (e.g. income, tenure, satisfaction scores) have the strongest raw relationship with someone leaving — an early hint at what will matter to the model.
Compare income & overtime against attrition	Used boxplots to compare monthly income between employees who left versus stayed, and computed attrition rate for overtime = Yes vs No.	Visually and numerically tests two of the most common real-world attrition drivers — pay and overwork — before assuming they matter.
Identify categorical columns	Listed all text/categorical columns and calculated how many columns one-hot encoding would produce.	Plans ahead for preprocessing — one-hot encoding expands each category into multiple binary columns, so it's worth knowing the resulting dataset width in advance.

3.2 Preprocessing & Train-Test Split

Raw data can't go directly into a model — numeric features are on different scales, and categorical text needs to become numbers. This stage handles both, and also sets aside a fair test set.

Step	What We Did	Why It Matters
Separate features and target	Removed employee_id and join_date from the feature set (they're identifiers, not predictive signal) and set attrition as the target, converted to 1/0.	employee_id is just a label with no real meaning, and using join_date directly risks leaking future information into the model — the problem statement specifically flags this as a data leakage risk.
Scale numeric features	Applied StandardScaler to all numeric columns (age, income, tenure, satisfaction scores, etc.), so each has mean 0 and standard deviation 1.	Models like Logistic Regression and K-Means are sensitive to feature scale — without this, a feature like income (thousands) would dominate a feature like education level (1–5) purely because of its larger numbers.
Encode categorical features	Applied OneHotEncoder (dropping the first category of each) to columns like department, job_role, gender, and business_travel.	Machine learning models need numbers, not text categories. Dropping the first level avoids redundant, perfectly-correlated columns (the 'dummy variable trap').
Combine both with ColumnTransformer	Wrapped the scaler and encoder together into a single ColumnTransformer, so numeric and categorical columns are each processed the correct way in one step.	Keeps preprocessing consistent and reusable — the same transformer can be applied identically to new, unseen data later.

Step	What We Did	Why It Matters
Train-test split with stratification	Split the data 80/20 into training and test sets, using stratify=y so the attrition ratio stays consistent in both sets.	Attrition is imbalanced (far more 'No' than 'Yes'). Without stratification, a random split could accidentally leave the test set with too few or too many leavers, distorting how trustworthy the evaluation metrics are.
Fit on train, transform on test only	The ColumnTransformer was fit only on the training data, then used to transform both the training and test sets.	This is a core rule of ML hygiene — the model (and its preprocessing) should never see the test set during fitting, or the evaluation becomes optimistic and unreliable.

3.3 Classification — Predicting Who Leaves

This is the core deliverable: a model that predicts attrition. Three different algorithms were trained so their strengths could be compared, rather than relying on just one.

Step	What We Did	Why It Matters
Logistic Regression	Trained a Logistic Regression model with class_weight='balanced' to account for the imbalance between leavers and stayers.	A simple, interpretable baseline — its coefficients are easy to explain to a non-technical HR audience, and 'balanced' weighting stops the model from just predicting 'No' for everyone to get a high accuracy score.
Decision Tree	Trained a Decision Tree classifier on the same data.	Captures non-linear patterns and interactions (e.g. 'high overtime AND low satisfaction') that a linear model like Logistic Regression can miss.
Random Forest	Trained a Random Forest — an ensemble of many decision trees.	Generally more accurate and stable than a single tree, and it also gives us feature importance scores, which become the backbone of the interpretability story.
Evaluate with the right metrics	For each model, calculated precision, recall, F1-score, and ROC-AUC rather than just accuracy.	With imbalanced classes, accuracy alone is misleading (a model that never predicts 'Yes' could still look 80%+ accurate). Precision/recall/F1 tell us how well the model actually catches real leavers versus false alarms.
Confusion matrix	Visualized true positives, true negatives, false positives, and false negatives as a heatmap.	Translates the model's mistakes into business terms: a false positive wastes retention budget on someone who wasn't leaving anyway; a false negative means a real flight risk goes unnoticed — arguably the costlier mistake.
ROC curves	Plotted ROC curves for all three models on one chart to compare them directly.	Gives a single visual for comparing how well each model separates leavers from stayers across all possible decision thresholds, not just one cutoff.
Feature importance	Extracted and ranked feature importances from the Random Forest model.	This is the 'why' behind the predictions — turns a black-box model into a list of the top 5–10 real-world factors (e.g. overtime, income, tenure, satisfaction) driving attrition, which HR can actually act on.

Step	What We Did	Why It Matters
Hyperparameter tuning (GridSearchCV)	Used GridSearchCV to test several values of the regularization parameter C for Logistic Regression and picked the best one via 5-fold cross-validation.	Rather than guessing a setting, this systematically finds the configuration that gives the best F1-score, and cross-validation makes that choice more robust than a single train/test split would.

3.4 Clustering — Segmenting Employees

Separate from prediction, this stage groups employees by their characteristics alone — without telling the algorithm who actually left — to see if natural, meaningful segments emerge.

Step	What We Did	Why It Matters
Elbow method	Ran K-Means for k = 2 through 10 clusters and plotted the resulting inertia (within-cluster distance) for each k.	Helps pick a sensible number of clusters — the 'elbow' point is where adding more clusters stops meaningfully reducing spread within each group.
Silhouette score	Computed the silhouette score for the same range of k values.	A second, independent check on cluster quality — measures how well-separated and cohesive the clusters are, to confirm (or challenge) what the elbow plot suggested.
Fit final K-Means model	Trained the final K-Means model using the chosen k and assigned every employee a cluster label.	Produces the actual employee segments that HR could use for differentiated policies — for example, a high-risk segment might warrant more frequent check-ins or compensation review.
Attrition rate by cluster	Grouped employees by their assigned cluster and calculated the attrition rate within each group.	This is where clustering becomes actionable — it reveals which specific segment of the workforce is leaving at the highest rate, so retention efforts can be targeted rather than company-wide.

3.5 Regression — Predicting Monthly Income

As a secondary exercise, the project also predicts monthly income from an employee's profile, to understand what drives compensation.

Step	What We Did	Why It Matters
Build regression dataset	Used all features except employee_id, join_date, attrition, and monthly_income_usd itself as predictors, with monthly_income_usd as the target.	Keeps the target out of its own feature set (that would be an obvious leak) and reuses the same clean feature list as the classification stage.
Linear Regression & Random Forest Regressor	Trained both a Linear Regression model and a Random Forest Regressor on the same data.	Linear Regression gives directly interpretable coefficients (dollars per unit change), while Random Forest usually captures more complex, non-linear salary patterns — comparing both shows the trade-off between interpretability and accuracy.
Evaluate with R^2 and RMSE	Measured both models using R^2 (variance explained) and RMSE (average dollar error).	R^2 tells us how much of the variation in income the model explains overall; RMSE translates

Step	What We Did	Why It Matters
		model error back into a real, interpretable dollar amount.
Interpret the strongest driver	Ranked the Linear Regression coefficients by magnitude to find the single feature most associated with higher income.	Turns the regression model into a business insight — e.g. quantifying how much job level or years of experience is 'worth' in dollar terms.

3.6 Pipeline & Reproducibility

Finally, the individual pieces were tied together into a structure that runs cleanly from start to finish and can be reapplied to new data.

- **Ordered execution:** load_data() → eda_plots() → preprocess_split() → fit_classification_models() → fit_kmeans() → fit_regression(), so the whole analysis can be reproduced in one run.
- **Saved outputs:** every chart (attrition breakdown, correlation heatmap, confusion matrices, ROC curves, elbow/silhouette plots, cluster attrition, feature importances) was saved to an output/ folder rather than only shown inline.
- **Pipeline object:** used sklearn's Pipeline to chain the scaler and the Logistic Regression model together, so both can be fit once and reused on brand-new employee data without ever refitting the scaler separately.

4. Summary of the Approach

Put together, the workflow answers the project's four objectives in order: EDA established what the data looks like and hinted at early drivers; preprocessing made the data model-ready without leaking information; three classification models — compared side by side — predict attrition with interpretable feature importances; clustering segments the workforce for targeted action independent of the prediction itself; and regression adds a secondary lens on what drives pay. Every stage was built to be reproducible, with all preprocessing fit strictly on training data and all charts saved for later reporting.

5. Recommended Next Steps

- Act on the top features flagged by Random Forest importance — for example, if overtime and low job satisfaction dominate, prioritize workload review and manager check-ins for those groups first.
- Use the highest-attrition cluster identified in the segmentation step to design a targeted retention program, rather than a one-size-fits-all policy.
- Revisit class-imbalance handling with SMOTE (from imblearn) if recall on the 'Yes' class needs to improve further beyond class_weight='balanced'.
- Periodically retrain the pipeline as new employee data comes in, using the same Pipeline object to avoid inconsistent preprocessing.